

Python Analysis of SMS Fiber-Optic Sensor Spectra

Whole-run spectral mapping, multi-fringe tracking and temperature calibration across the critical wavelength

Technical report and reproducibility guide

Author: Kuzey Kaya

Dataset: smsdata20241017.csv — SMS sensor cooldown run, 17 October 2024

Code: sms-fiber-analysis (Python package `sms_analysis` + scripts), delivered as `for_advisor.zip`

Date: 29 September 2026

Version: 1.0

Abstract

This report documents a Python pipeline that replaces the group's Igor Pro workflow for single-mode-multimode-single-mode (SMS) fiber-sensor spectra. The pipeline loads a run of transmission spectra, detects every peak and dip in every spectrum, tracks each interference fringe individually over time with guards against fringe-hopping, fits a wavelength-versus-temperature line per fringe, and renders the whole run as a two-dimensional wavelength-time map. Applied to a 730-spectrum cooldown from 44 °C to 22.8 °C, it tracks 22 of 24 fringes over the full run ($R^2 \geq 0.90$, most ≥ 0.997), measures sensitivities from -88 to -320 pm/°C below the critical wavelength ($\lambda_c \approx 1525$ nm) and from $+115$ to $+403$ pm/°C above it, and reproduces the group's published value (-141 vs. -142 pm/°C near 1493–1495 nm). The report places the work in the dispersion-turning-point and whole-spectrum-demodulation literature, explains why the temperature run cannot yield the staircase graph of the strain experiment, and gives the commands that regenerate every figure and table.

Contents

1. Introduction and literature context
2. Data
3. Methods and code — loading, detection, tracking, calibration, figures
4. Results — the whole run in one image; sensitivity across the spectrum; calibration
5. Peak inventory and interval sampling
6. The staircase question
7. Reproducibility — commands, outputs, package layout
8. Discussion and next steps
9. References

Figures 1–11 and Tables 1–3 are regenerated from the raw CSV by the scripts named in Table 3.

1. Introduction and literature context

An SMS fiber sensor is a short length of multimode (here few-mode) fiber spliced between two single-mode fibers. Light excites several modes in the multimode section; at the output they interfere, so the transmission spectrum is a fringe pattern $T(\lambda) = t_1 + t_2 + 2\sqrt{t_1 t_2} \cdot \cos \Delta\phi(\lambda)$, where $\Delta\phi$ is the phase difference between the interfering modes. Temperature and strain change $\Delta\phi$, so the fringes shift; the sensor reads out as a wavelength shift of a chosen peak or dip [2] [6].

With a few-mode fiber the spectrum shows a **critical wavelength** λ_c , the dispersion turning point at which the group-index difference of the interfering modes vanishes. Fringes widen toward λ_c , and the sensitivity of a fringe depends on its distance from λ_c : it grows toward λ_c and changes sign across it [1] [3] [8]. The 2023 review by Xu et al. surveys sensors that exploit this turning point in microfiber couplers, gratings and in-fiber interferometers, and reports sensitivities up to 16.78 nm/°C; it also names the price — narrow operating bandwidth, spectral broadening of the fringes, and the difficulty of extracting the signal from noisy spectra [4].

That last point, demodulation, is the open problem this report addresses. Conventional readout follows one dip; once a shift exceeds one free spectral range the fringe identity becomes ambiguous and single-dip tracking fails. Two 2025 responses use the whole spectrum instead of one feature: Hu et al. train an LSTM on the full transmission spectrum and extend the usable range to about three free spectral ranges [5], and Salik et al. recover the phase difference between two wavelengths by phase unwrapping, reaching more than 7000 $\mu\epsilon$ with sensitivity intact [6]. Chen et al. show that a reflective probe version of the sensor keeps the same temperature sensitivity, reporting -0.142 nm/°C near 1495 nm [7].

Position of this work. The pipeline described here is the data layer those whole-spectrum methods need: it treats a run as a wavelength–time map, tracks every fringe simultaneously rather than one, measures sensitivity versus wavelength directly instead of assuming it, and is open, scripted and reproducible on any machine. It reproduces the published Igor analysis on the same kind of data and is the base on which the phase-unwrapping method and a user interface can be built.

2. Data

The acquisition writes one CSV per run: column 0 is temperature (°C), column 1 is time (s), and every further column is the transmittance at the wavelength named in its header (nm). Each row is one full spectrum. The run analysed here, [smsdata20241017.csv](#), holds 730 spectra from 1475 to 1575 nm at 0.1 nm steps (1001 wavelength columns), recorded every ≈ 23 s during a continuous cooldown from 44.1 °C to 22.8 °C over 4.8 h. Transmittance ranges from 0.26 to 0.83. The input is continuous — no setpoint steps — which matters for Section 6.

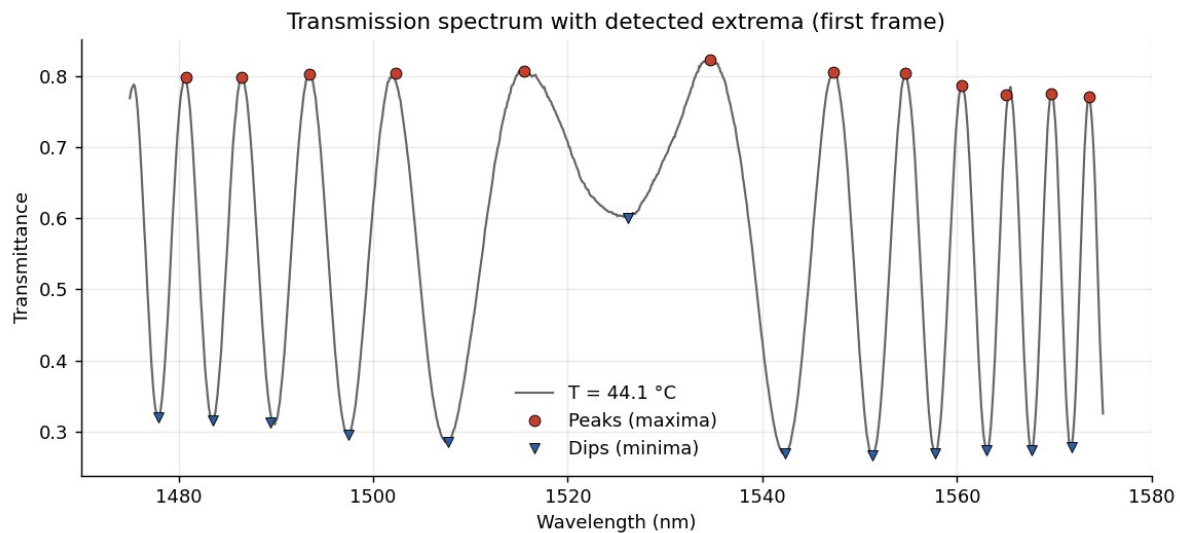


Figure 1. First spectrum of the run (44.1 °C) with the detected peaks (circles) and dips (triangles). Fringes are dense at the band edges and widen around the critical wavelength near 1525 nm. Produced by `scripts/run_analysis.py`.

3. Methods and code

The code is a small Python package, `sms_analysis`, plus command-line scripts. Each step below names the module and function that implements it so that the report can be read alongside the code.

3.1 Loading — `sms_analysis/io.py`

`load_sms_csv(path)` reads the CSV with `pandas`, parses the wavelength headers, and returns a `SpectraSet` (arrays `wavelengths`, `time`, `temperature`, and the 730×1001 matrix `spectra`). Any file in the same layout loads unchanged.

3.2 Peak and dip detection — `sms_analysis/peaks.py`

`detect_extrema(wavelengths, spectrum, prominence=0.02, distance=5, smooth_window=11)` applies a Savitzky-Golay filter (window 11 samples, order 3) and finds maxima and minima with `scipy.signal.find_peaks` [9]. The prominence threshold of 0.02 sits between the fringe depth (> 0.05) and the noise (≈ 0.005). Each extremum is then refined to sub-sample precision by `parabolic_refine`, which fits a parabola through the extremum and its two neighbours; a guard rejects the vertex when $|\text{offset}| > 1$ sample (a nearly flat neighbourhood), because the unguarded formula can throw the position by several nanometres on noisy data — the cause of the step in the original notebook (Section 6).

3.3 Fringe tracking — `sms_analysis/tracking.py`

`track_feature(data, start_wavelength, kind, search_window=1.5, max_step=0.5)` follows one peak or dip frame by frame inside a ± 1.5 nm window around its last position, refining on the smoothed curve. Two guardrails keep it on the same fringe: a hit at the window edge means the window no longer contains a true extremum (the fringe has faded or merged) and is recorded as lost lock; and any jump larger than 0.5 nm between consecutive frames is rejected, since a real fringe moves well under 0.1 nm per 23 s frame. `track_all_extrema(data)` runs this for every extremum found in the first spectrum — 24 fringes in parallel. Tracking noise on a well-behaved fringe is about ± 30 pm.

Two further trackers exist for comparison and for the staircase question: `track_feature_legacy` reproduces the original Colab algorithm exactly (4 nm window, unguarded refinement on raw data, no rejection), and `track_band_extremum` follows whichever fringe is strongest inside a fixed band, so its trace hops from fringe to fringe.

3.4 Calibration — `sms_analysis/analysis.py`

`fit_sensitivity(track)` fits wavelength against temperature with `scipy.stats.linregress` and returns the slope (sensitivity, reported in pm/°C), its standard error and R^2 .

`sensitivity_map(tracks)` tabulates this for every fringe.

3.5 Figures — `sms_analysis/plotting.py`, `igor_style.py`, `staircase.py`, and the scripts

All figures are matplotlib [10]. `plotting.py` holds the analysis figures; `igor_style.py` renders the tracking graph in the group's Igor Pro house style (red wavelength on the left axis, blue temperature on the right, 'Time, seconds'); `staircase.py` averages a track inside temperature bins and draws the plateaus. The scripts are: `run_analysis.py` (the main pipeline, figures 1–7 and the sensitivity table), `peak_inventory.py` (exports every extremum of every spectrum, the trajectory map and the interval-sampled staircase), `hero_figures.py` (the annotated spectral maps) and `spectral_3d.py` (the 3-D views).

4. Results

4.1 The whole run in one image

Figure 2 is the central result: the full run as a wavelength–time map with every tracked fringe overlaid. Fringes below λ_c drift to longer wavelength on cooling, fringes above λ_c to shorter; the drift grows toward λ_c ; and the fringes between about 1515 and 1535 nm broaden into the central band and are lost within the first 40 minutes. Figure 3 re-indexes the same map by measured temperature: the exponential cooldown straightens out and each fringe becomes a straight line whose slope is its sensitivity. Figure 4 shows the three regions enlarged.

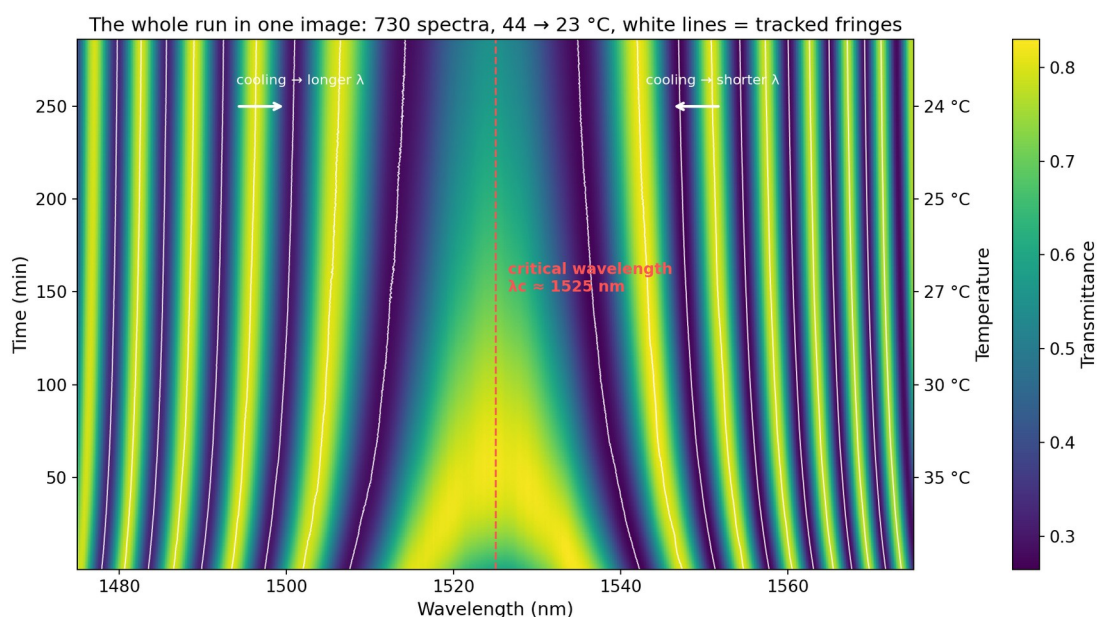


Figure 2. Spectral map of the full run: wavelength (x), time (y, temperature on the right axis), transmittance (colour). White lines are the 22 tracked fringes; the red dashed line marks $\lambda_c \approx 1525$ nm. Produced by scripts/hero_figures.py.

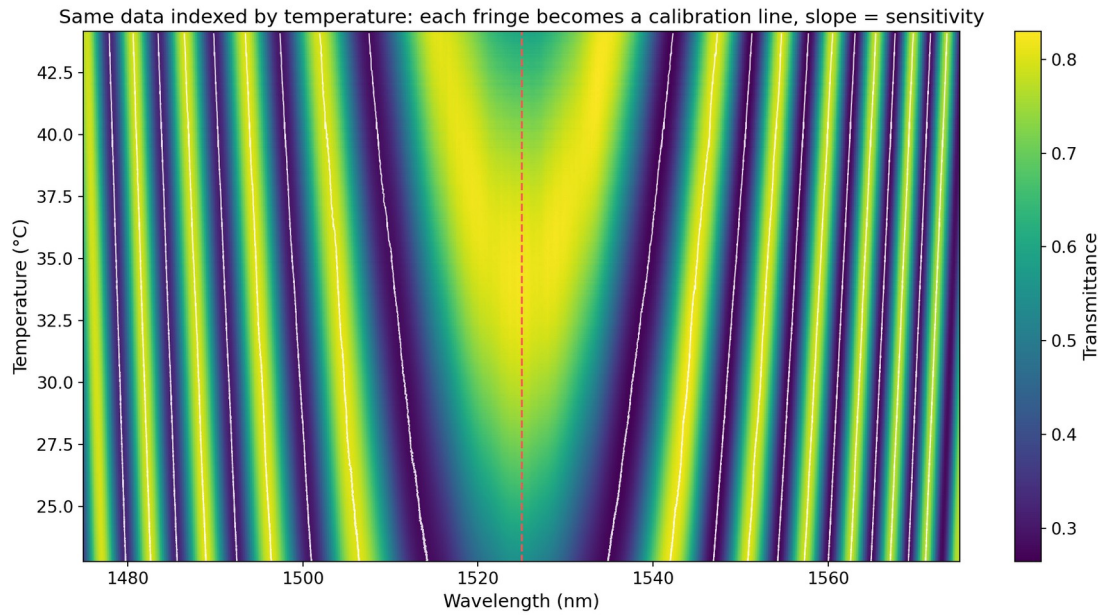


Figure 3. The same data indexed by temperature. Each white line is one fringe's calibration line $\lambda(T)$; its slope is that fringe's sensitivity. Produced by scripts/hero_figures.py.

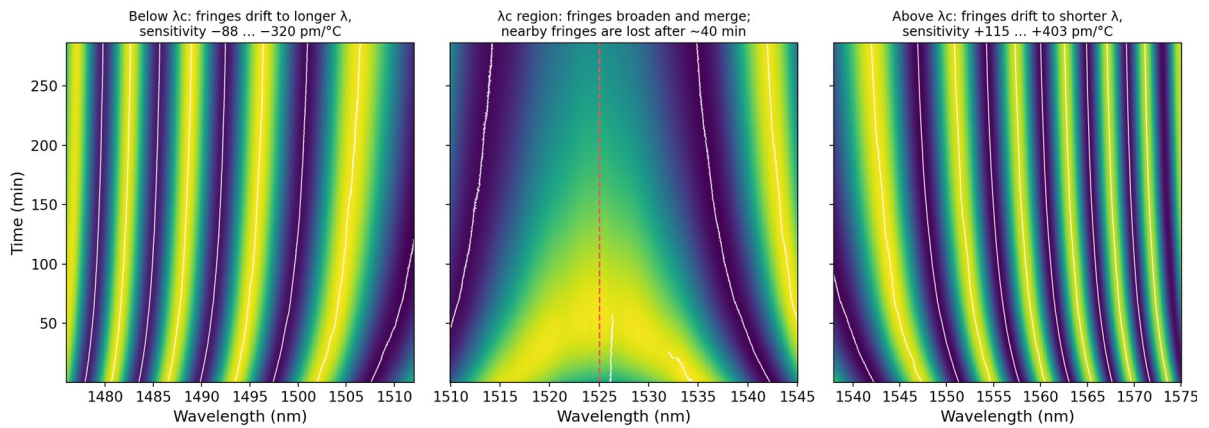


Figure 4. Three regions of the map: below λ_c , the λ_c region, above λ_c . Produced by scripts/hero_figures.py.

Fringe landscape: transmittance as height over wavelength $\times$ temperature
ridges = peaks, valleys = dips; they tilt in opposite directions across λ_c

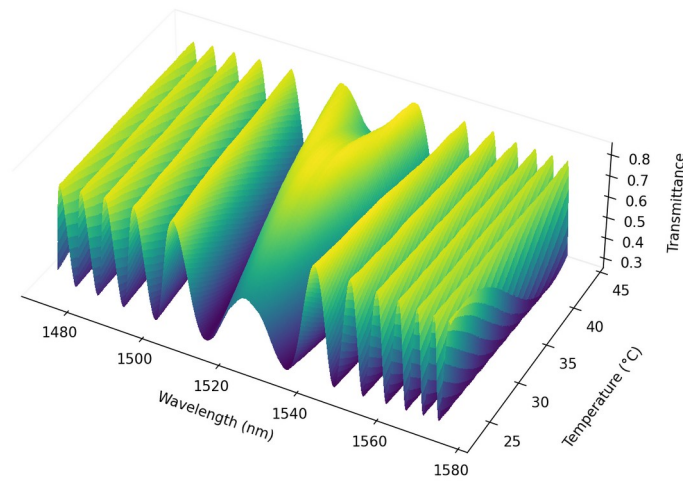


Figure 5. Fringe landscape: transmittance as height over wavelength $\times$ temperature. Ridges are peaks, valleys are dips; they lean in opposite directions across λ_c and merge into the broad hump at λ_c . Produced by scripts/spectral_3d.py (an interactive, rotatable HTML version is written when plotly is installed).

4.2 Sensitivity across the spectrum

Twenty-two of the 24 fringes present in the first spectrum are tracked over all 730 frames with $R^2 \geq 0.90$ (most ≥ 0.997); the two exceptions are the fringes closest to λ_c , which merge into the central band. Table 1 lists every fringe with its start and end wavelength, total drift, sensitivity and R^2 . Sensitivity is negative below λ_c (−88 to −320 pm/°C) and positive above it (+115 to +403 pm/°C), and its magnitude grows toward λ_c from both sides — the behaviour described in [1] [3] and shown for this sensor type in Fig. 3(b) of [7]. As a cross-check against the group's Igor analysis, the fringe starting at 1493.4 nm gives −141.3 pm/°C; the published value near 1495 nm is −142 pm/°C [7].

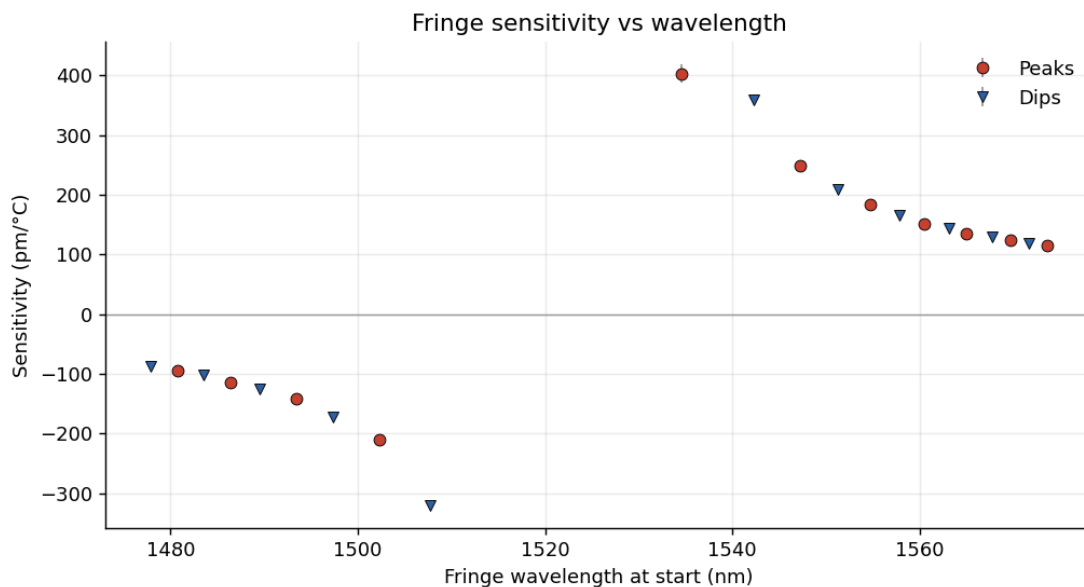


Figure 6. Sensitivity of every tracked fringe versus its wavelength at the start of the run (error bars: standard error of the slope). Produced by scripts/run_analysis.py.

Table 1. Direction and sensitivity of every tracked fringe (from figures/fringe_directions.csv). 'Locked' is the number of frames, out of 730, in which the tracker held the fringe.

Fringe	Start (nm)	End (nm)	Shift (nm)	Sensitivity (pm/°C)	R ²	Locked
dip@1477.9	1477.9	1479.8	1.86	-87.9	1.000	730
peak@1480.6	1480.6	1482.6	1.98	-94.4	0.999	730
dip@1483.5	1483.5	1485.7	2.17	-102.5	1.000	730
peak@1486.4	1486.5	1488.9	2.45	-115.2	0.999	730
dip@1489.8	1489.8	1492.5	2.64	-124.8	1.000	730
peak@1493.4	1493.5	1496.4	2.94	-141.3	0.998	730
dip@1497.4	1497.4	1501.0	3.60	-172.7	0.999	730
peak@1501.9	1501.9	1506.5	4.52	-209.6	0.994	730
dip@1507.7	1507.6	1514.2	6.61	-320.3	0.996	730
peak@1515.7	1515.7	1515.8	0.14	-145.8	0.661	9
dip@1526.2	1526.2	1526.4	0.16	-21.3	0.604	132
peak@1534.6	1534.6	1532.1	-2.49	403.4	0.911	65
dip@1542.3	1542.3	1534.9	-7.43	359.3	0.997	730
peak@1547.2	1547.3	1542.0	-5.36	248.5	0.997	730
dip@1551.3	1551.3	1546.9	-4.38	209.8	1.000	730
peak@1554.7	1554.7	1550.8	-3.86	184.1	0.999	730
dip@1557.8	1557.8	1554.2	-3.51	166.2	1.000	730
peak@1560.5	1560.5	1557.3	-3.22	151.2	1.000	730
dip@1563.1	1563.1	1560.0	-3.05	143.4	1.000	730
peak@1565.4	1565.4	1562.6	-2.86	134.3	1.000	730
dip@1567.7	1567.7	1564.9	-2.75	129.3	1.000	730
peak@1569.7	1569.7	1567.1	-2.62	123.7	1.000	730
dip@1571.7	1571.7	1569.2	-2.52	118.9	1.000	730
peak@1573.6	1573.6	1571.1	-2.44	115.4	1.000	730

4.3 Calibration of the most sensitive trackable fringe

The dip starting at 1542.3 nm is the most sensitive fringe that stays locked for the whole run: $+359.3 \pm 0.7$ pm/°C, $R^2 = 0.997$ over 21 °C (Figure 7). The slight curvature at the ends of the fit is physical — as the fringe cools it moves away from λ_c and its sensitivity decreases a little — and is consistent with the sensitivity-versus-wavelength trend in Figure 6. A piecewise or quadratic fit is available if a single-number sensitivity is not the goal.

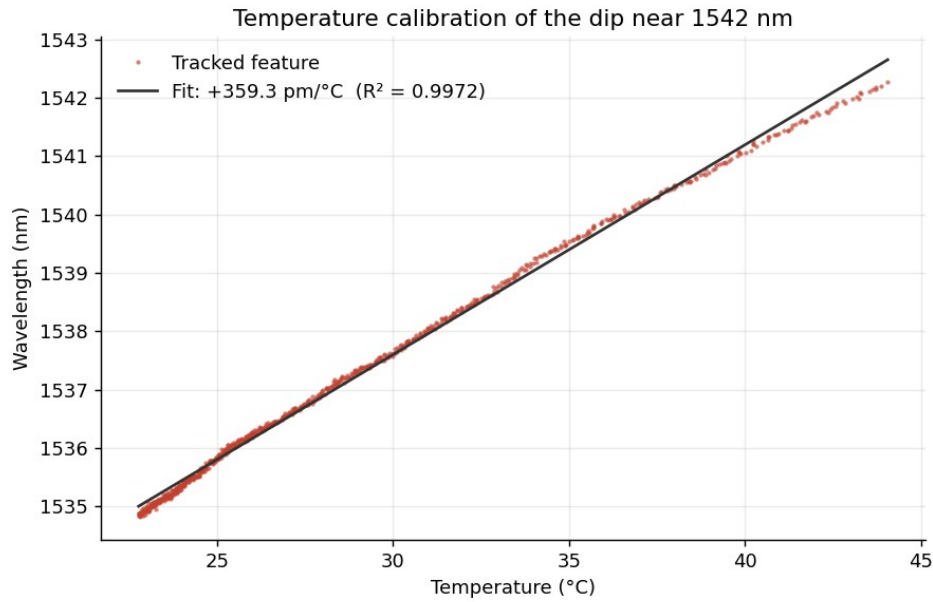


Figure 7. Tracked wavelength of the dip starting at 1542.3 nm versus temperature with the linear fit. Produced by `scripts/run_analysis.py`.

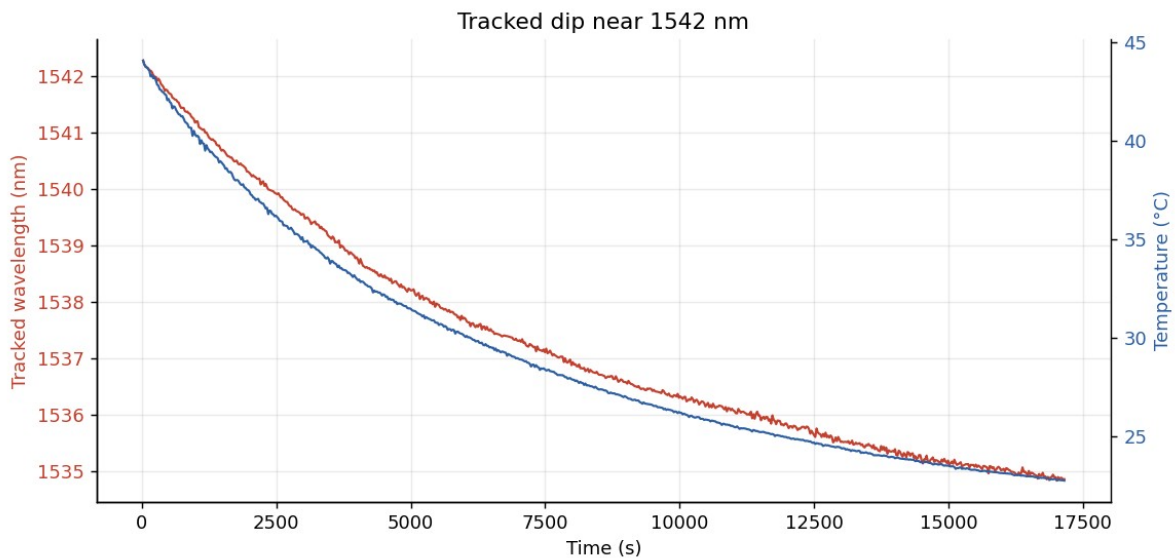


Figure 8. The same fringe (red, left axis) and temperature (blue, right axis) against time — the monitoring view used in the group's Igor graphs; an Igor-style rendering (fig7) is in the `figures` folder. Produced by `scripts/run_analysis.py`.

5. Peak inventory and interval sampling

Following the advisor's procedure for slowly varying data, `scripts/peak_inventory.py` exports every extremum of every spectrum to `all_peaks_long.csv` (17,665 rows: frame, time, temperature, kind, wavelength, value) and the identity-resolved version `all_peaks_tracked_wide.csv` (one column per fringe). Figure 9 plots all extrema against time; each streak is one fringe's trajectory and the direction of drift is read directly. Sampling a track every 1200 s ($\approx 1^\circ\text{C}$) and holding the value in between turns the slow drift into a staircase (Figure 10); binning by temperature and averaging gives the plateau table (Table 2), whose rows can serve as calibration points.

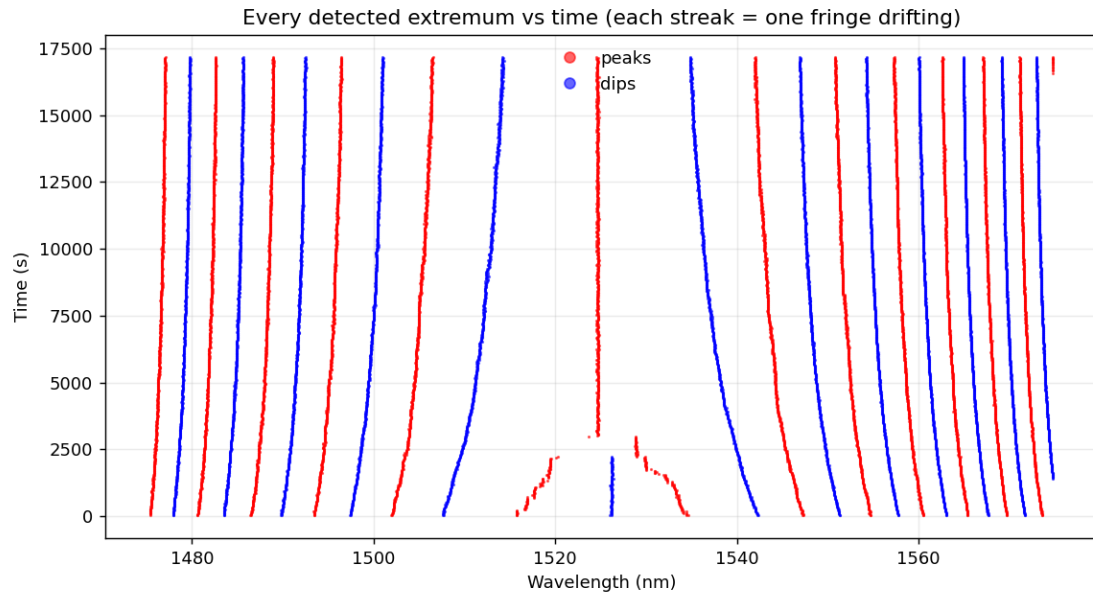


Figure 9. Every detected extremum of every spectrum versus time (red: peaks, blue: dips). Produced by `scripts/peak_inventory.py`.

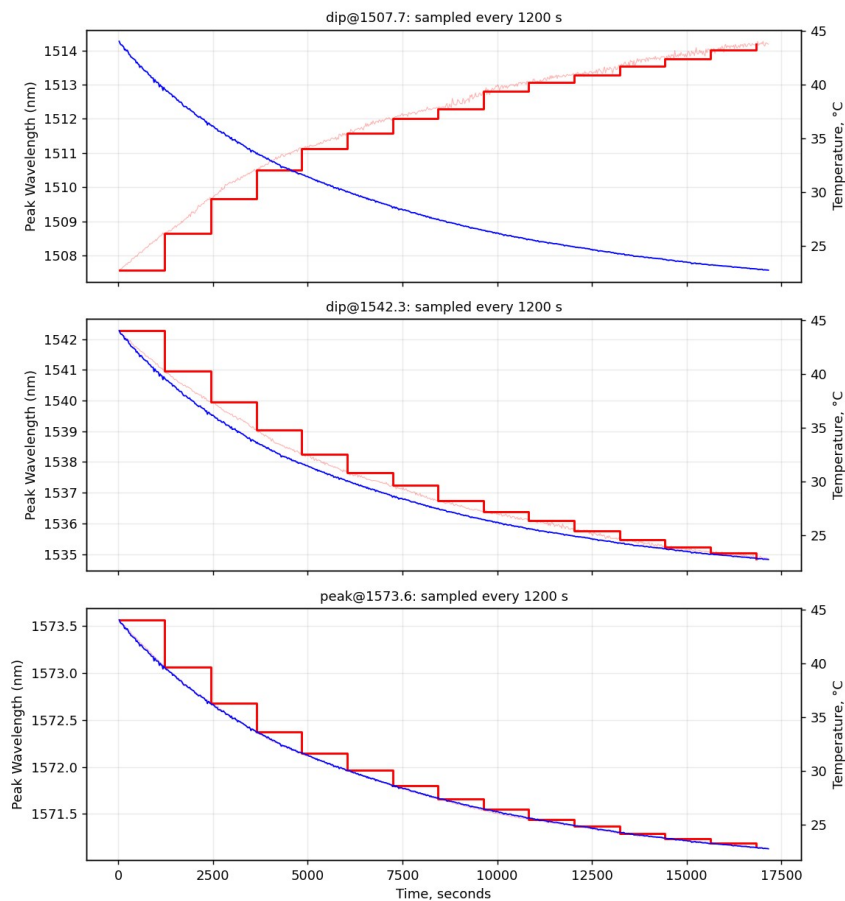


Figure 10. Three tracked fringes sampled every 1200 s and held between samples (bold), over the raw tracks (faint): one below λ_c (rising), one just above (falling), one at the band edge. Produced by `scripts/peak_inventory.py --interval 1200`.

Table 2. Plateau averages of the 1542.3 nm dip in 3 °C temperature bins (figures/staircase_plateaus.csv).

t start (s)	t end (s)	Mean T (°C)	Mean λ (nm)	$\sigma\lambda$ (nm)	Points
23	496	43.00	1541.976	0.170	21
520	1412	40.45	1541.230	0.277	39
1436	2539	37.45	1540.307	0.255	48
2562	3995	34.43	1539.325	0.321	62
4018	6111	31.38	1538.160	0.317	90
6135	9002	28.38	1537.085	0.317	123
9026	13842	25.34	1535.956	0.351	206
13866	17156	23.34	1535.090	0.144	141

6. The staircase question

The group's slide 'Peak tracking: peaks near the critical wavelength are more sensitive' shows the peak wavelength climbing in steps from 1543 to 1559 nm while the temperature stays flat at about 20 °C, with labels 0 g, 50 g, 250 g and 300 g. That graph comes from the strain experiment of [6]: 50 g added every 5 minutes, temperature constant. Each step is a new load and each plateau a hold time; the staircase is the sensor's physical response to a stepped input.

The cooldown run analysed here has a continuous input, so every fringe drifts smoothly and no zoom level reveals steps. Step-shaped traces can be produced from it in two ways, and both were implemented so the distinction is explicit: by sampling or binning the track (Section 5), where the steps are a presentation of the smooth drift; or by letting the tracker hop between fringes. The large step in the original notebook's graph (Figure 11) is of the second kind: the unguarded parabolic refinement on raw data occasionally throws the position by several nanometres, the 4 nm search window teleports with it, and the tracker settles in the deep trough near λ_c . The step height, about 11 nm, is one fringe spacing, not a jump of the sensor. The current tracker (Section 3.3) rejects exactly this, which is why the same fringe now tracks for 730 of 730 frames with $R^2 = 0.9998$.

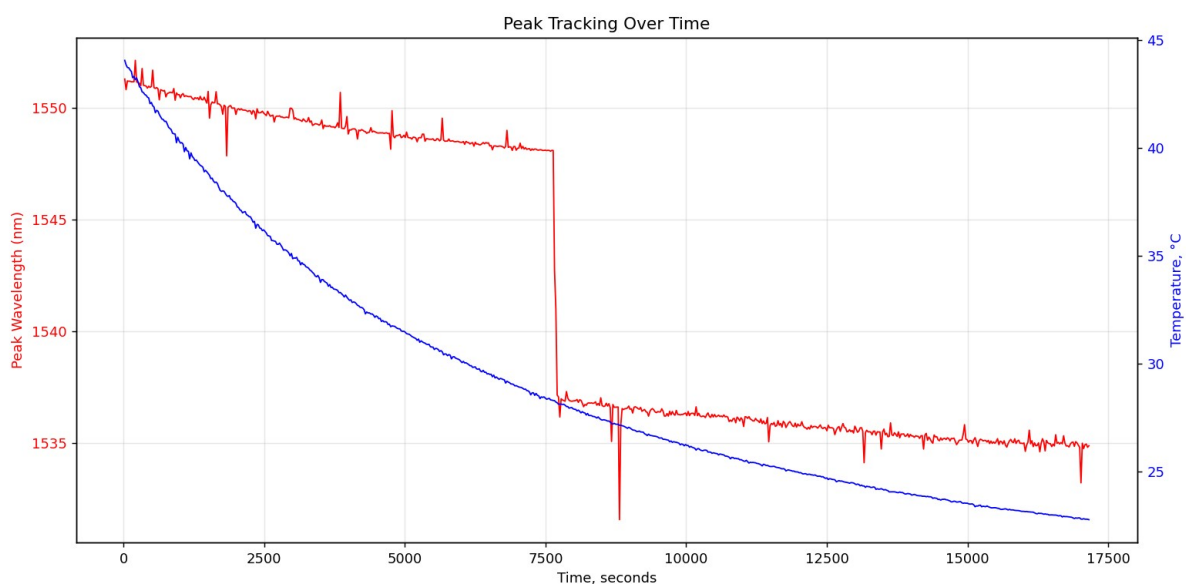


Figure 11. Exact reproduction of the original Colab tracking on this run (`track_feature_legacy`): the tracked dip hops from the 1551 nm fringe into the λ_c trough at $t \approx 7660$ s. Produced with scripts and functions in the package; see README.

If the strain experiment's CSV is available in the same layout, the pipeline reproduces the slide's graph without any change to the code.

7. Reproducibility

Requirements: Python ≥ 3.10 with numpy, pandas, scipy and matplotlib ([requirements.txt](#)); plotly is optional for the interactive 3-D export. On macOS, from the unzipped `sms-fiber-analysis` folder with the CSV placed in `data/`:

```
pip3 install -r requirements.txt
python3 scripts/run_analysis.py data/sms-data-2024-10-17.csv # figs 1-7, sensitivity_map.csv, tracked_feature.csv
python3 scripts/peak_inventory.py data/sms-data-2024-10-17.csv --interval 1200 # all peaks, fig13-14
python3 scripts/hero_figures.py data/sms-data-2024-10-17.csv # figs 15-17
python3 scripts/spectral_3d.py data/sms-data-2024-10-17.csv # figs 18-19 (+ interactive HTML with plotly)
open figures
```

On Windows use `python` and `pip` instead. The scripts must run from the project folder (they import the `sms_analysis` package next to them); copying a single script elsewhere breaks the import. Useful options: `--track <nm>` and `--track-kind peak|dip` choose the fringe for the detailed plots, `--prominence` sets the detection threshold, `--search-window` the tracking window, `--interval` the sampling interval of the staircase view.

Table 3. Outputs of the pipeline and where each figure in this report comes from.

Output	Script	Content
fig1_spectrum_peaks.png	run_analysis.py	First spectrum with detected extrema (Fig. 1)
fig2_spectra_overlay.png	run_analysis.py	Spectra at five temperatures
fig3_spectral_map.png	run_analysis.py	Wavelength–time heatmap of the run
fig4_tracking_vs_time.png	run_analysis.py	Tracked fringe + temperature vs time (Fig. 8)
fig5_correlation.png	run_analysis.py	λ vs T with linear fit (Fig. 7)
fig6_sensitivity_map.png	run_analysis.py	Sensitivity of every fringe vs wavelength (Fig. 6)
fig7_igor_style.png	run_analysis.py	Tracking graph in Igor Pro style
fig9_staircase*.png	run_analysis.py / staircase.py	Temperature-bin plateaus of the tracked fringe
fig10_legacy_*.png	tracking.track_feature_legacy	Original Colab tracking reproduced (Fig. 11)
fig12_*.png	tracking.track_band_extremum	Strongest-fringe-in-band tracking (fringe hops)
fig13_peak_trajectories.png	peak_inventory.py	All extrema vs time (Fig. 9)
fig14_interval_staircase*.png	peak_inventory.py	Interval-sampled tracks (Fig. 10)
fig15-17	hero_figures.py	Annotated map, map vs temperature, zooms (Figs. 2–4)
fig18-19, fig18_3d_interactive.html	spectral_3d.py	3-D views (Fig. 5); HTML needs plotly

Output	Script	Content
sensitivity_map.csv	run_analysis.py	Per-fringe sensitivity table
tracked_feature.csv	run_analysis.py	Time, T, λ , value of the detailed fringe
all_peaks_long.csv / all_peaks_tracked_wide.csv	peak_inventory.py	Every extremum of every spectrum; identity-resolved wide table
fringe_directions.csv	peak_inventory.py	Per-fringe drift direction and sensitivity (Table 1)
staircase_plateaus.csv	run_analysis.py / staircase.py	Plateau means (Table 2)

Package layout. `sms_analysis/io.py` (loading), `peaks.py` (detection and refinement), `tracking.py` (four trackers), `analysis.py` (fits), `plotting.py`, `igor_style.py`, `staircase.py` (figures); `scripts/` (the four entry points above and a clearly labelled simulation demo); `README.md` (usage and the full figure table).

8. Discussion and next steps

Three things the pipeline changes in practice. First, the analysis becomes a whole-run operation: one command turns a CSV into the map, the multi-fringe calibration and the tables, in about three minutes, with no manual peak picking. Second, sensitivity versus wavelength is measured on every run, so the choice of reference fringe can be made from data — the fringe closest to λ_c that still tracks over the intended temperature range, here the 1542.3 nm dip. Third, fringe-hopping is detected rather than silently followed, which removes the largest error source of single-dip tracking on data that crosses a fringe spacing [5].

Planned next steps: (i) a small interface on top of the pipeline — load a CSV, choose a fringe, obtain the maps, the calibration and the sensitivity table; (ii) porting the phase-unwrapping method of [6] onto this data layer, so that the wide-dynamic-range readout is available alongside peak tracking; (iii) a stepped-temperature run (setpoint held and changed in steps), which would give a cleaner calibration than the continuous cooldown; and (iv) reproducing the strain-experiment graph directly from its CSV.

Open questions for the advisor: which dataset produced the staircase slide and whether its CSV can be shared; the exact meaning of 'tracking at intervals' (time intervals, temperature intervals, or a widening search window in Igor); which fringe should serve as the reference (the Igor graph title names peak1546); and how Igor defined a peak (raw extremum, or after smoothing or fitting), so that the two analyses can be compared number by number.

9. References

- [1] E. Salik, M. Medrano, G. Cohoon, J. Miller, C. Boyter, J. Koh, "SMS fiber sensor utilizing a few-mode fiber exhibits critical wavelength behavior," *IEEE Photonics Technology Letters* 24(7), 593–595 (2012). <https://ieeexplore.ieee.org/document/6138899/>
- [2] J. Zhou, B. He, X. Gu, "Transmission spectrum characteristics for a single-mode-multimode-single-mode fiber filter," *IEEE Photonics Technology Letters* 26(21), 2185–2188 (2014).
- [3] C. Lu, J. Su, X. Dong, L. Lu, T. Sun, K. T. V. Grattan, "Studies on temperature and strain sensitivities of a few-mode critical wavelength fiber optic sensor," *IEEE Sensors Journal* 19(5) (2019).

- [4] S. Xu, P. Kang, Z. Hu, W. Chang, F. Huang, "Ultrasensitive optical fiber sensors working at dispersion turning point: review," *Sensors* 23(3), 1725 (2023). <https://doi.org/10.3390/s23031725>
- [5] J. Hu, S. Zhang, M. Cai et al., "LSTM-assisted optical fiber interferometric sensing: breaking the limitation of free spectral range," *Light: Science & Applications* 14, 392 (2025). <https://www.nature.com/articles/s41377-025-02008-4>
- [6] E. Salik, T. Nguyen, S. Chen, Y. Cong, D. Pitones, A. Ozer, "Single-mode-multimode-single-mode fiber sensor with wide dynamic range," *Optica Sensing Congress*, paper SM1E.2 (2025).
- [7] S. Chen, D. Gonzalez, K. Vandervoort, E. Salik, "Single-mode-multimode-single-mode fiber sensor probe with reflective gold coating," *Optica Sensing Congress*, paper JM4A.8 (2025).
- [8] N. Irawati, L. Gruner-Nielsen, L. S. Rishoj, K. Rottwitt, "Strain and temperature sensor using few-mode fiber, designing guidelines and results," *Optical Fiber Technology* 80, 103465 (2023).
- [9] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods* 17, 261–272 (2020). Used for peak detection (`scipy.signal.find_peaks`), Savitzky–Golay smoothing and linear regression. <https://doi.org/10.1038/s41592-019-0686-2>
- [10] J. D. Hunter, "Matplotlib: a 2D graphics environment," *Computing in Science & Engineering* 9(3), 90–95 (2007). Used for all figures. <https://doi.org/10.1109/MCSE.2007.55>

Note on tooling

The code was restructured from the author's earlier Colab notebook into the present package with the assistance of Claude (Anthropic). All experimental data, the target analyses, the choice of methods and the interpretation of results are the author's and the group's; every figure in this report is regenerated from the raw CSV by the scripts named in Table 3.